



Six projects that prove you can do the job

A build spec for engineers pivoting into AI roles. Six named projects, each one a real company problem, each one with a ship gate you can hold yourself to.

The rule: a project only counts when it has an eval, a number, and a tradeoff you can defend out loud.

How to use this

- Pick one. Six half finished repos read worse than one finished repo with a results table.
- Build the eval before the feature. If you cannot measure it, you cannot claim it in an interview.
- Keep the use case. Recruiters remember Groundtruth, the eval harness for a law firm. They forget project four.
- Write the tradeoff down. The recommendation is the part hiring managers actually read.
- New to this? Start with project 04. It is the shortest, and it makes every other project on this list measurable.

The six

01	Fieldnote	multimodal rag field service assistant for HVAC techs	intermediate two weekends
02	Casefile	multi agent orchestration claims triage for a regional insurer	advanced two weekends
03	Warmstart	llm caching support assistant at 400k questions a month	intermediate one weekend
04	Groundtruth	retrieval evaluation eval harness for a legal research assistant	foundational one weekend
05	Slotfill	lora finetuning freight invoice extraction at 40k docs a month	advanced two weekends
06	Doorman	prompt injection guardrails recruiting agent that reads candidate uploads	intermediate one to two weekends



Fieldnote

multimodal rag

level: intermediate effort: two weekends

the use case

A commercial HVAC contractor runs 40 field technicians. The knowledge they need is split across 300 equipment manuals full of wiring diagrams and pressure curves, 4,000 photos of failed parts uploaded from job sites, and recorded calls with the manufacturer support line. A tech standing on a roof asks why unit 7 keeps tripping on high pressure. The answer lives in a chart on page 112, not in a paragraph.

what it proves

You can retrieve across text, images and audio in one system, and answer questions whose evidence only exists inside a picture.

the build

- Ingest the mixed corpus: manual PDFs with charts and tables, job site photos, support call recordings.
- Index each modality with its own encoder instead of captioning everything into text. Keep a modality tag on every chunk.
- Fuse at retrieval time: run parallel searches, then rerank the merged candidate set with a cross encoder.
- Return the source page image with the answer so the tech on the roof can verify the number themselves.

stack

SigLIP or CLIP for images, Whisper for audio, a text embedding model, pgvector or Qdrant with modality filters, a vision capable model for final synthesis

ship gate

- 40 question eval set, 15 of which are unanswerable from text alone
- Beat a caption-only baseline on those 15 by a margin you can show in a table
- Every answer carries a citation with page number and a cropped image
- p95 latency under 6 seconds at k=20

what it signals

You can explain early fusion versus late fusion, and why converting a chart to a caption throws away the axis values the question actually needed.

the trap

Captioning every image with a vision model, embedding the captions, and calling it multimodal. That is a text pipeline wearing a costume.



Casefile

multi agent orchestration

level: advanced effort: two weekends

the use case

A regional auto insurer receives 900 claims a day. Each one needs documents extracted, prior claims and policy limits checked, damage estimates sanity checked against repair costs, and a recommendation written. One prompt cannot hold that. Four coordinated agents can, as long as the system stops when it should and never pays out without a human.

what it proves

You can decompose a task across specialised agents and keep the system bounded, observable and replayable.

the build

- A supervisor plus three specialists: extractor, investigator, reviewer. Typed handoffs, no free text passed between nodes.
- Shared claim state in an external store so any run can be resumed from a snapshot.
- A budget ceiling per claim in tokens and dollars, plus a max step count that terminates the graph.
- A human approval gate in front of any payout recommendation or write back to the claims system.

stack

LangGraph or a hand rolled state machine, Redis or Postgres for state, JSON schema tool contracts, OpenTelemetry for traces

ship gate

- 30 recorded claim runs, each with a trace showing the exact node path taken
- One run replayed from a stored snapshot, reaching the same terminal state
- The reviewer sends work back at least once and the graph still terminates
- Cost per claim charted, with the ceiling enforced by code and not by hope

what it signals

You talk about termination conditions, loop guards and cost ceilings before you talk about agent personas. That is the line between an engineer and a prompt hobbyist.

the trap

Agents that hand off in a circle with no stop condition. The demo works once and the bill arrives later.



Warmstart

llm caching

level: intermediate effort: one weekend

the use case

An ecommerce brand runs an AI support assistant that answers 400,000 questions a month. Roughly six in ten are the same handful of questions phrased differently: where is my order, how do returns work, does this ship to Canada. They are paying frontier model prices to answer the same question repeatedly, and p95 latency is hurting conversion on the checkout page.

what it proves

You can cut inference spend and tail latency while proving you did not start serving wrong answers.

the build

- Three layers: exact match, semantic match above a tuned similarity floor, and provider side prefix caching for the static system block.
- A cache key that includes model, temperature, tool set, customer tier and system prompt version. Version it or you will serve yesterday's behaviour.
- TTL plus explicit invalidation on catalogue reindex, policy change and prompt change.
- A hit rate and savings dashboard the support lead can read without you in the room.

stack

Redis, an embedding model for semantic keys, provider prompt caching, a replay harness over logged traffic

ship gate

- Replay 1,000 production shaped queries and report hit rate, cost delta, and p50 and p95 latency delta
- Measure false hit rate with a judge model on a sample of semantic hits, and keep it under 1 percent
- Show the precision curve as you move the similarity threshold
- Prove that a system prompt change busts the cache

what it signals

You understand that a semantic cache is a retrieval system with a precision problem, and that threshold tuning is a business decision about how wrong you can afford to be.

the trap

Keying the cache on the user question alone. Two customers with different order histories get the same answer, and you have shipped a data leak instead of an optimisation.



Groundtruth

retrieval evaluation

level: foundational effort: one weekend

the use case

A 60 attorney firm has a research assistant over 20 years of case files and filings. Every week someone changes a chunk size, swaps an embedding model or adds a reranker, and nobody can say whether last month's version was better. A missed precedent is a malpractice conversation, so opinion is not an acceptable form of evidence.

what it proves

You can prove a retrieval change was an improvement instead of asserting it.

the build

- A golden set of 100 query and passage pairs mined from real filings, reviewed by a human, not invented by a model in one shot.
- Retrieval metrics: recall@k, MRR and nDCG, reported per query category so failures are legible.
- A generation layer on top: faithfulness and answer relevance, kept separate from the retrieval numbers.
- A regression harness in CI that fails the pull request when recall@10 drops by more than a point.

stack

Ragas or custom scorers, pytest, a versioned dataset file, a simple comparison report

ship gate

- Three configurations compared: chunk size, hybrid versus dense, reranker on and off
- A results table plus a written recommendation with the tradeoff stated
- The CI gate wired and demonstrated failing on a deliberately bad change
- A short error analysis naming the five queries that still fail, and why

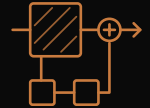
what it signals

You separate retrieval failure from generation failure. Most teams cannot, which is why their systems plateau at demo quality.

the trap

Eyeballing five queries, feeling good, and shipping. That is a demo, and an interviewer can tell in one question.

Slotfill



lora finetuning

level: advanced effort: two weekends

the use case

A freight brokerage processes 40,000 carrier invoices a month across hundreds of layouts. A frontier model extracts them well but costs more than the clerks it replaced, and schema violations still break the accounting import twice a week. The question is whether a small tuned model can hold the schema at a fraction of the cost.

what it proves

You know when a fine tune beats a prompt, and you can ship an adapter that serves in production.

the build

- Lock the task to a hard correctness signal: extraction into a strict invoice schema that either validates or does not.
- Build 1,000 or more real labelled examples with a documented labelling process, then hold out 200 that never touch training.
- Train a LoRA adapter on a small open model. Log rank, alpha, learning rate and one ablation you actually ran.
- Serve the adapter and compare it against the prompted frontier baseline on the same held out set.

stack

PEFT and transformers, or Axolotl or Unsloth, one rented GPU, vLLM with adapter loading for serving

ship gate

- Schema validity rate and field level accuracy for adapter versus prompted baseline, same test set
- Cost per 1,000 invoices and p95 latency for both paths
- A clear recommendation, including the honest outcome where the prompt wins and you ship the prompt
- Eval numbers, never training loss, as the headline result

what it signals

You frame finetuning as a cost, latency and control tradeoff. Anyone can run a notebook. Few can say when not to.

the trap

Training on 200 synthetic examples generated by the same model family you are trying to beat, then reporting a falling loss curve as evidence.



Doorman

prompt injection guardrails

level: intermediate effort: one to two weekends

the use case

A recruiting platform runs an agent that reads candidate submitted resumes and portfolio PDFs, scores them, emails candidates and writes status back to the applicant tracking system. Every document it reads is written by someone with a direct incentive to manipulate the outcome, and white text at the bottom of a resume is a five second attack.

what it proves

You can reason about a system where the input is hostile and the model is gullible.

the build

- An agent that reads candidate uploaded documents and holds real tools: email, scoring, ATS writes. That is the attack surface.
- Layered defence: input classification, hard isolation of untrusted content, a per context tool allowlist, output scanning, and confirmation on irreversible actions.
- A red team suite of 60 attacks, including injection hidden in resume metadata and injection returned inside a tool result.
- Structured logs naming the rule that fired on every block.

stack

A small classifier or guard model, a policy layer in front of the tool router, an attack corpus you maintain yourself

ship gate

- Attack success rate before and after the guardrails, reported by attack family
- False positive rate on 100 benign resumes, reported next to it
- One documented bypass you found and then closed
- Every block traceable to a rule in the log

what it signals

You know that filtering the phrase about ignoring previous instructions is theatre, and that the real control is limiting what the agent is allowed to do once it is fooled.

the trap

A system prompt that politely asks the model not to fall for tricks, shipped as a security control.

Build these with people doing the same thing

Post your ship gate results inside the community, get them torn apart by working AI engineers, and fix them before an interviewer does.

join

<https://www.skool.com/baswe-ai/about>